The Cheat Code

Agent Patterns for Copilot Chat

⇑ ⇑ ⇓ ⇓ ⇐ ⇒ ⇐ ⇒ B A START

ISSUE #003 • APRIL 6, 2026 • WEEKLY

Last issue we fixed noisy search. This issue tackles the next problem: getting an agent to *reason* through a multi-step process instead of trying to do everything in a single prompt.

This issue's patterns: How to chain prompts so each step improves the next, and how to define those steps in YAML playbooks that anyone on the customer's team can modify without touching code.

🎮 AGENT SPOTLIGHT

The Work Item Triage Agent (continued)

Built by Raghav BN • Part 2 of 3 from this agent

The scenario: Your customer has an intake process. Requests come in — IT tickets, HR inquiries, legal cases, procurement approvals. Someone reads each one, checks if it's complete, looks up related context, classifies it, and decides where to route it. It's 15–30 minutes of cognitive work per request, and it happens dozens of times a day.

The single-prompt approach ("summarize and classify this request") fails here because the task requires *sequential reasoning*. The agent needs to first understand what the request is about, then search for context, then assess quality, then classify, then recommend. Each step builds on the previous one.

🧩 PATTERN BREAKDOWN: PART A

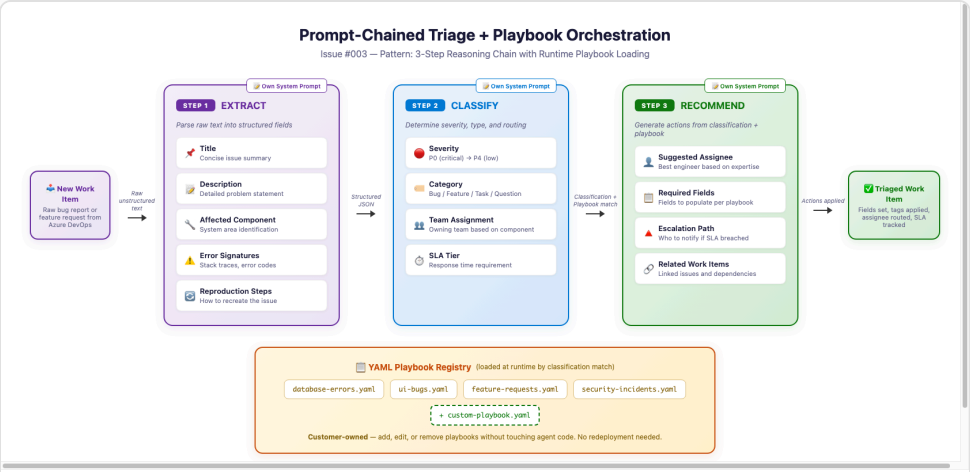
Prompt-Chained Triage

Step-by-step agent reasoning instead of single-shot prompts

Why single-shot fails: When you ask an LLM to "analyze this and tell me what to do," it tries to be helpful all at once. The result is broad but shallow — it summarizes but doesn't truly classify, it suggests but doesn't cross-reference, it answers but doesn't catch what's missing.

The pattern: Decompose the task into a chain of discrete prompts. Each step has a single job and produces a structured output that feeds the next step. The agent follows a deliberate reasoning path.

```
Step 1: Read input → extract structured fields
                ↓
Step 2: Generate scoped search queries
                ↓
Step 3: Search sources → aggregate context
                ↓
Step 4: Classify type + assess completeness
                ↓
Step 5: Recommend actions with evidence
```



Prompt chain + playbook orchestration — each step narrows the problem space for the next

🔬 **Explore this pattern interactively →**

The search in step 2 is more precise *because* step 1 extracted structured fields. The classification in step 4 is more accurate *because* step 3 found relevant history. Each step narrows the problem space for the next.

🧩 **PATTERN BREAKDOWN: PART B**

Playbook-Driven Orchestration

Separate what to do (YAML) from how to execute (agent)

The problem prompt chaining creates: Once you have a multi-step reasoning chain, someone needs to define and maintain those steps. If the process logic is hardcoded in the agent, only developers can change it. The customer's process owners — the people who actually know the triage rules — are locked out.

The pattern: Define the prompt chain in a YAML playbook. The playbook describes *what* steps to execute and what each step should produce. The agent's orchestration layer handles *how* — executing prompts, passing context between steps, managing errors. This separation means:

Playbook (YAML) — owned by process team

```
step_1: Read work item, extract fields
step_2: Search {sources} for {keywords}
step_3: Classify into {categories}
step_4: Generate recommendations
```

Agent orchestrator — owned by engineering

```
Executes steps, chains context, handles errors
```

Why CSAs should lead with this:

- **New process?** New playbook. No code changes, no dev sprints.
- **Multiple teams?** Each team gets their own playbook. Same agent.
- **Governance?** Playbooks are versionable, reviewable, auditable — like any config file.
- **Demos?** Swap playbooks to show different scenarios without rebuilding anything.

Customer conversation starter: "What if your process owners could define how the agent handles requests — without filing a dev ticket every time the rules change?"

🛠️ **TRY THIS NOW**

Map Your Customer's Intake Process to a Playbook

1. **Pick the intake process.** IT tickets, HR requests, legal case intake, procurement approvals — find one where someone reads, classifies, and routes 10+ items per day.
2. **Shadow the triage person.** Write down every step they take: what they read, where they search, what they check for, how they classify, where they route. That's your prompt chain.
3. **Translate to a playbook.** Each human step becomes a YAML step with a clear input, instruction, and expected output. Share it with the process owner for validation — they can read YAML even if they can't write code.
4. **Show the split.** Demonstrate that the playbook can change without the agent changing. Swap in a different playbook for a different team's process. Same agent, different behavior. This is what gets the customer excited about scalability.

💡 **Positioning:** "The agent is the engine. The playbook is the steering wheel. Your team decides where it goes — without calling IT every time the route changes."

🏆 **WHERE THESE PATTERNS LAND**

- **IT service desks** — ticket classification, priority assignment, SLA routing
- **HR operations** — employee request intake, benefits routing, policy questions
- **Legal intake** — case classification, document completeness, precedent search
- **Procurement** — vendor request triage, approval routing, contract review
- **Any team with a queue** — if humans read, classify, and route, this pattern applies

> **The pitch that lands:** "Show me your highest-volume intake process. I'll map it to a playbook this week, and you'll have an agent handling triage by next week. When the process changes, your team updates the playbook — no engineering required."

Next issue: Secure In-Boundary Processing

The architecture for customers who say "our data can't leave our tenant."

The Cheat Code

ABS Tech Strategy • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Browse all issues at aka.ms/the-cheat-code

📦 Code sample for this issue

Microsoft Internal • Not for external distribution